Machine learning project report

Predicting the french electricity consumption from weather data

Félix de Batz

March - April 2026

Table des matières

1 Introduction **3**

2 Dataset creation **3**
- 2.1 Data Acquisition . . . 3
- 2.2 Filtering the data . . . 4
- 2.3 Merging the datasets . . . 6
- 2.4 Analysing the datasets . . . 6
- 2.5 Enhancing our datasets . . . 8

3 Training and evaluating our models **9**
- 3.1 Splitting our datasets . . . 9
- 3.2 Evaluating our models . . . 9

4 The models **10**
- 4.1 Polynomial regression . . . 10
- 4.2 Ridge . . . 14
- 4.3 Decision trees . . . 19
- 4.4 AdaBoosting our trees . . . 24
- 4.5 First summary . . . 28

5 New datasets ? **28**
- 5.1 Creating the Departments dataset . . . 28
- 5.2 Introduction the PCA . . . 29

6 Going back to the region datasets **33**

7 Testing our models on the new datasets **34**
- 7.1 The Daily dataset . . . 34
- 7.2 Hourly Datasets . . . 38

8 Filling the holes **38**
- 8.1 Interpolate the missing values . . . 38
- 8.2 First idea . . . 38
- 8.3 Nearest Neighbors . . . 38

9 The filled datasets's results **39**

10 Final summary **40**
- 10.1 For the Day prediction . . . 40
- 10.2 Hourly dataset . . . 44

11 Conclusion **48**
- 11.1 Comparing against RTE prediction . . . 48
- 11.2 What's next . . . 49

12 Annexes **51**
12.1 Feature names explained 51

1 Introduction

Energy management, particularly in the electricity sector, has become a global priority as nations strive for energy sovereignty [5]. A significant challenge in transitioning to renewable energy sources is the difficulty of large-scale storage, which necessitates precise distribution timing. Consequently, accurately forecasting electricity consumption is essential for effectively managing and planning national production.

In this project, we ask the question : To what extent can machine learning techniques predict French electricity consumption ?

According to our research on the *RTE* (Réseau de Transport d'Électricité) website - the company responsible for French electricity distribution - current forecasting models rely heavily on features such as meteorological data and public holidays to estimate D-1 consumption [3]. Leveraging the accessibility of comprehensive weather datasets in France, this study aims to develop a predictive model for electricity demand based on these environmental and temporal variables.

2 Dataset creation

2.1 Data Acquisition

We plan to combine weather data with French electricity consumption statistics.

2.1.1 Weather Data

Météo France provides open access to meteorological data through the *data.gouv* platform. This portal includes records from all French weather stations, available at both daily [2] and hourly [1] resolutions.

For the hour dataset, we only kept the 2020-2024 period, as it already represents more than 40,000 rows. Moreover, we can see that the hourly dataset has more features than the daily one.

The data is partitioned by department, but it remains easily accessible for download. After downloading and extracting all the required files, we end up with around 24 gigabytes of data, most of which is irrelevant for our task. We will see how we dealt with that in the next point.

Technical Note : API Performance

There is an API available to directly download the relevant rows and columns, but it was far too slow for our requirements. Even after optimization, the process would have taken dozens of hours.

2.1.2 Electricity Consumption Data

Electricity distribution is managed by *RTE*, which provides extensive datasets on its website [3]. Notably, they publish their own day-ahead predictions [4] ; at some point we will compare our model's performance against these official forecasts. Although regional-level data is only available from 2013 onwards, it offers a high temporal resolution of thirty minutes. Again, we can simply download it and extract the .xls files. It was fully complete, so we did not have to deal with missing values. **The only region missing is Corsica ; we will not consider it in our project.**

2.2 Filtering the data

2.2.1 Weather data

The first part of the filtering is done on the date ; as for the day dataset, we have all data from 1950. Since our consumption data only starts at 2013, we delete all data previous to the 1st of January, 2013. The data given by *Météo France* is extremely rich but has a lot of sparse or redundant columns. To keep our model precise and not too computer-intensive, we only select certain columns based on two criteria : their relevance and their fullness over all weather stations.

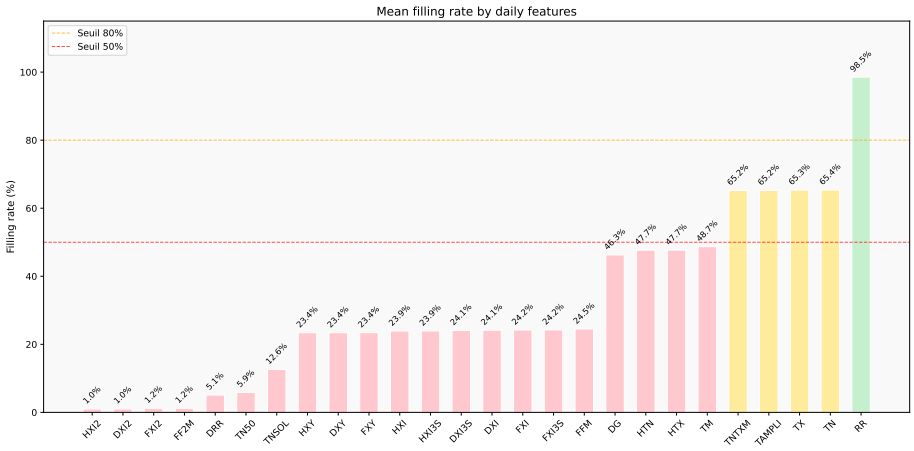


FIGURE 1 – Filling rate of daily features

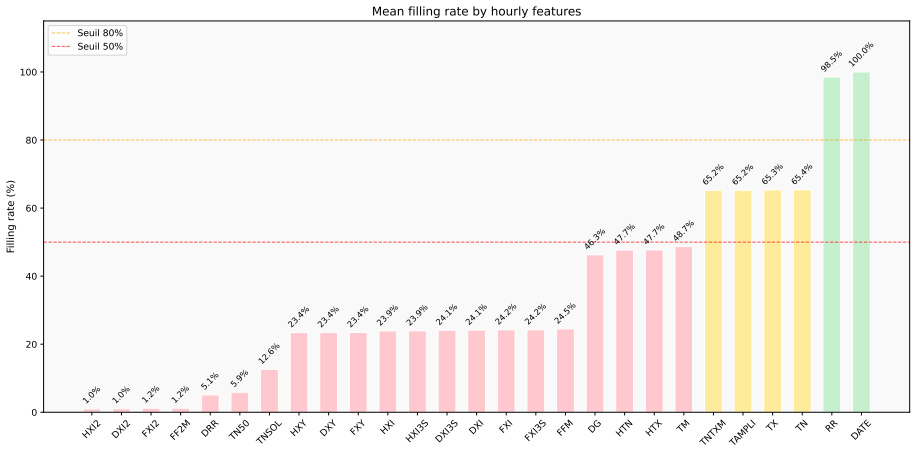


FIGURE 2 – Filling rate of hourly features

As we can see, most of the features are highly incomplete ; hopefully the most important features are reported quite well. To see the significance of all features, please refer to the day description 12.1.1 and hour description 12.1.2 in Annexe. We decided to keep the following features in our datasets :

TABLE 1 – Comparison of selected features for Daily and Hourly datasets

Daily Features	**Hourly Features**	**Significance**
RR	RR1	Precipitation amount
FFM	FF	Mean wind speed at 10m
TM	T	Temperature (mean daily vs. hourly)
TN	TN	Minimum sheltered temperature
TX	TX	Maximum sheltered temperature
DG	DG	Frost duration (daily vs. minutes)
-	TD	Dew point temperature
-	U / UN / UX	Humidity (current, min, max %)

2.2.2 Electricity Consumption Data

On the *RTE* dataset, we are only interested in the consumption and not the production or exchange part, so we can simply drop all irrelevant columns, leaving us with two features, the datetime and the consumption in the region. Then we can sum consumption by hour to unify with the weather data.

2.3 Merging the datasets

We now have three distinct categories of datasets : one with weather data by station and day, another one by hour, and the last one with electricity consumption by region.

2.3.1 The Region-Day datasets

Here we want to create a dataset to find the consumption by day. Firstly, we sum the electricity consumption to end up with daily consumption. We then add to deal with the weather data. Keeping all features from all stations was not our first idea, as it seemed extremely computer intensive ; to simplify our task and ease the computer cost, we computed the mean value for each feature on a region scale, ignoring the missing values. So we end up with 12 datasets, one for each region. Each of them has 4,383 rows and 8 columns.

2.3.2 The Region-Hour datasets

Here we want to create a dataset to find the consumption by hour, hoping it will be more precise than the daily one. For the same reason, we could not keep all station information. To represent our weather data, we also compute the mean on the region scale, ignoring the missing values. So we end up with 12 datasets, one for each region. Each of them has 43,848 rows and 17 columns.

2.4 Analysing the datasets

Let's take a look at our data to see what we are dealing with. We first show the electricity consumption compared to the mean temperature on a daily scope.

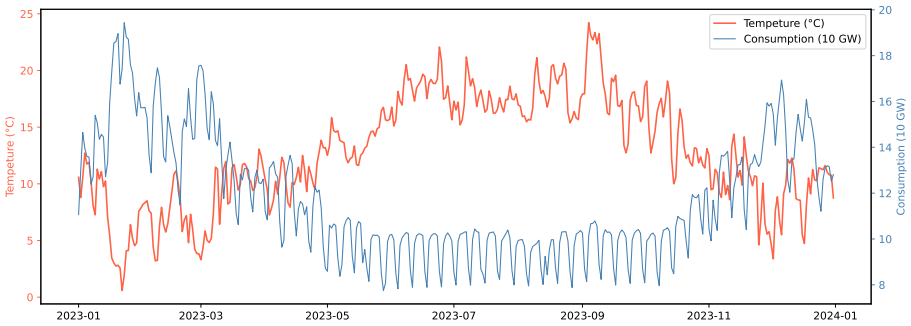


FIGURE 3 – Temperature VS consumption in Bretagne in 2023

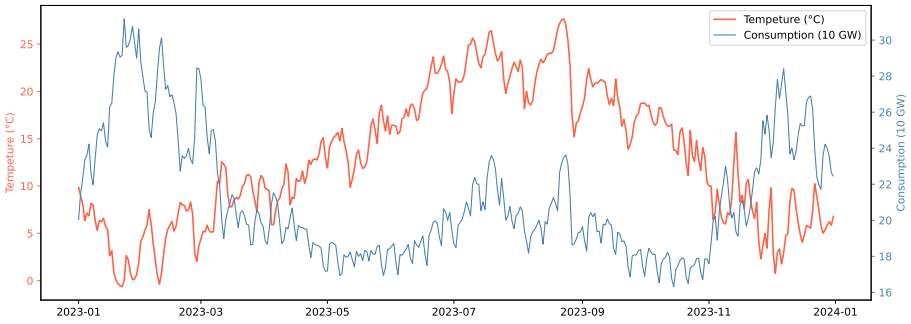


FIGURE 4 – Temperature VS consumption in PACA in 2023

On those graphics, we see the temperature and consumption of two regions during 2023 ; we only show those two graphics as they encapsulate the most interesting points. We can clearly see that a correlation between the two lines exists. An unexpected event occurs during the summer in Provence-Alpes-Côtes d'Azur (PACA)4, as the electricity consumption is positively correlated with the temperature. In contrast to the rest of the year, this phenomenon appears in all southern regions and is most probably due to the usage of climatization. Another surprising phenomenon appears in the summer of Bretagne3 : the consumption drops down at regular intervals. We first thought it was some kind of miscalculation or missing data. But after a deeper analysis, we realize it happened all Sunday, as we can see in the next plot.

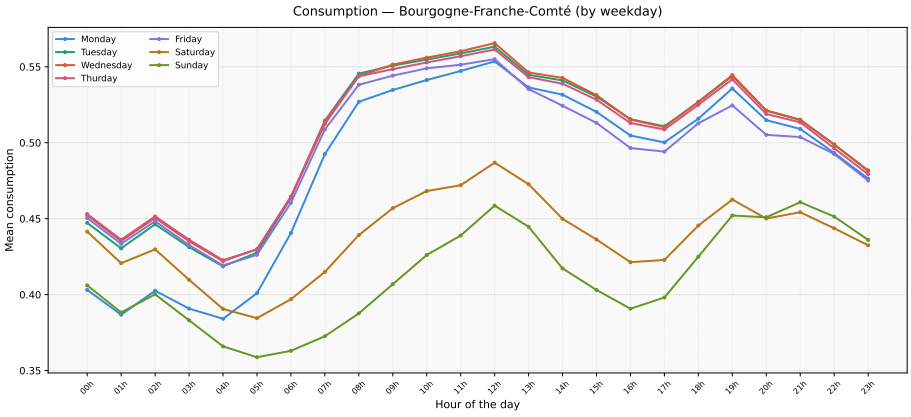


FIGURE 5 – Mean consumption by hour in Bourgogne-Franche-Comté

This graphic perfectly illustrates the importance of the hour and day of the week in electricity consumption. We only show one region, but we obtain

the same type of graph for every region. We can see two main things : first, the electricity consumption drops during the night before rising back up at dawn. This is particularly important since the night is often colder than the day, meaning that at a day scale the correlation observed on the previous graphic is no longer true. The second important point to notice is that Sunday's and Saturday's consumption are systematically lower than the rest of the week ; knowing the day of the week is important for our model to properly predict the electricity consumption.

2.5 Enhancing our datasets

As we already mentioned above, we realize that the day of the year, day of the week, and hour have an important role in electricity consumption. To encapsulate those periodic features, we use a sin-cos encoding. We added at the start of our pipeline a preprocessing phase ; it first extracts the values of the weekday, year day, and hour and then converts those columns into a sin-cos encoding.

As the electricity consumption is continuous, knowing the last consumption could be a great source of information for our model, and it is still compatible with our goal as we know last days' or hours' consumption when looking for tomorrow's one. So we added a function to add the last consumptions to the features, as it modifies the size of the dataset ; we could not implement it into a pipeline. The number of consumption points to consider will be optimized when fine-tuning the models.

TABLE 2 – Engineered features for Daily and Hourly datasets

Daily Features	**Hourly Features**	**Significance**
Year_Day$_{sin}$	Year_Day$_{sin}$	Yearly seasonality (Sine of the day of the year)
Year_Day$_{cos}$	Year_Day$_{cos}$	Yearly seasonality (Cosine of the day of the year)
Week_Day$_{sin}$	Week_Day$_{sin}$	Weekly cycle (Sine of the day of the week)
Week_Day$_{cos}$	Week_Day$_{cos}$	Weekly cycle (Cosine of the day of the week)
-	Hour$_{sin}$	Daily cycle (Sine of the hour)
-	Hour$_{cos}$	Daily cycle (Cosine of the hour)
Consommation_N	Consommation_N	Lag feature (Consumption recorded N time steps prior)

3 Training and evaluating our models

3.1 Splitting our datasets

To properly test, fine-tune, and fit our models, we need to split our dataset into two parts, the train set and the test set. We wanted to keep at least a full year in the test set, and it should represent 20% of our data. To make sure our model doesn't see the future while it is training, we decided to **split our daily dataset at January 1, 2022** and to **split our hourly dataset at January 1, 2023**.

To fine-tune our hyperparameters, we will use cross-validation splits on the training set (D_{train}) with the following parameters :

TABLE 3 – Cross-validation parameters

Parameters	**Daily dataset**	**Hourly dataset**
number of splits	2	2
gap	11	50
max train size	3,000	30,000
test size	670	8,000

We choose to limit the number of splits to only two to be able to explore more parameter combinations while keeping the computational cost reasonable. We also made sure that at least a full year was considered in the D_{valid} datasets.

Technical Note : Cross-validation on time series-related data

The datasets are time-related ; we have to take care of data leaks. In other words, make sure our models do not see the future while they are training. To avoid data leakage, we use a special cross-validation for time series where the training data is always before the validation data, and we even added a gap at least as large as the last consumption to make sure there is no data leak between the validation and training sets.

3.2 Evaluating our models

Our process to evaluate a model m was as follows :

1. **Manually** define Θ_m^1, a broad set of values for the hyperparameters.
2. Use a **grid search with time series cross-validation** to find different sets of good parameters.
3. If needed, define Θ_m^2, a narrower set of values for fewer hyperparameters to optimize.

4. Recompute a **grid search** to find the best parameters $\theta_m^* \in \Theta_m^2$ and record relevant computational information.

5. Fit m on D_{train} using θ_m^* as hyperparameters, then compute the score on D_{test}.

4 The models

As we are looking to predict a continuous value, we oriented ourselves to regression models. We implemented them in Python using the sklearn library. We implemented the following models :

1. Polynomial regression
2. Ridge regression
3. Decision tree
4. Decision tree with AdaBoosting

Every time, we train and optimize on all regions independently. Since the result tables are too big, we only show in this report the average obtained by all regions ; if there are big disparities between regions, we will mention it in the analysis.

4.1 Polynomial regression

The first model we explored was polynomial regression. We implemented it in a pipeline chaining polynomial features, then linear regression.
This method introduces a hyperparameter, polynomial degree $d \in \mathbb{N}$. We also tried to optimize the number of last consumptions to consider c.

4.1.1 The Region - Day dataset

$$\Theta^1_{\text{polyReg}} = \begin{cases} d \in \{1, 2, 3, 4\} \\ c \in \{0, 3, 7, 11\} \end{cases} \tag{1}$$

Here is the table of the mean of all regions with all tested parameters on the Day-Region dataset.

TABLE 4 – Mean gridsearch results for all regions on Day-Region dataset for Polynomial regression

d	c	Test Score	Train Score	Fit Time	Test Time
1	0	0.8034	0.8473	0.0310	0.0200
	3	0.9444	0.9452	0.0249	0.0250
	7	0.9507	0.9548	0.0216	0.0277
	11	0.9580	0.9629	0.0251	0.0226
2	0	0.8917	0.9288	0.0614	0.0252
	3	0.9748	0.9794	0.0994	0.0242
	7	0.9753	0.9818	0.1867	0.0382
	11	0.9745	0.9836	0.2156	0.0366
3	0	0.8856	0.9450	0.2457	0.0224
	3	0.9681	0.9861	0.6912	0.0340
	7	0.9303	0.9910	2.8767	0.0513
	11	-0.3194	0.9965	15.7355	0.0540

We were quite impressed to achieve such good results on our very first methods, with the average test score over 0.97.
As we can see, the polynomial degree chosen has a big impact on the performance of our model, and 2 seems to be the sweet spot ; 1 is a bit faster but less precise, 3 tends to overfit, and at 4 (not shown here) it overfits even more. We can also see that knowing the last consumption has a big impact, but the number of last consumptions has only a small impact, as it barely shifts the mean test score.

Another important piece of information is the coefficients of the features. We use a StandardScaler in our pipeline, and then we normalize the coefficients. We only show the top 10 features with the parameters of the best score i.e $d = 2$ and $c = 7$.

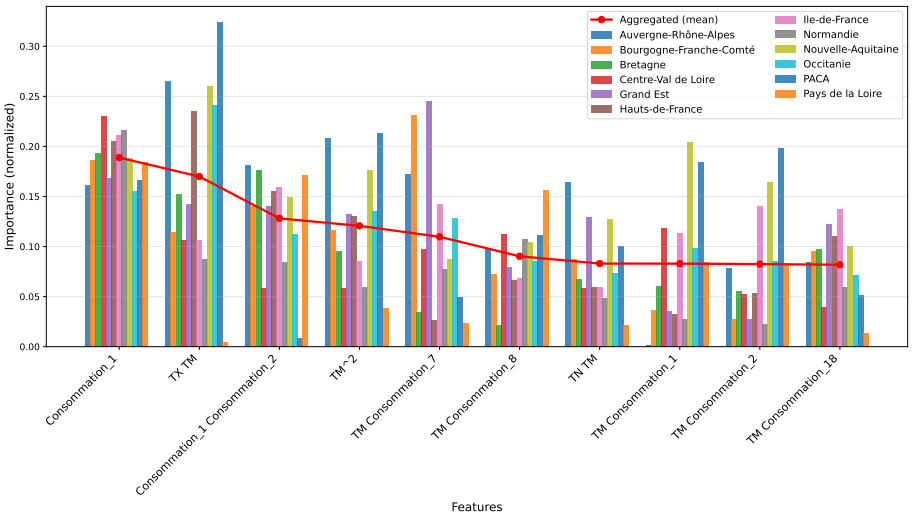


FIGURE 6 – Top daily feature importance for polynomial regression ($d = 2, c = 7$)

As our intuition thought, knowing the last consumptions is highly valued by the model ; it also relies a lot on the temperature. Overall it looks like each region has its own important features.

4.1.2 The Region - Hour dataset

And here is a section of a similar table for the Hour-Region dataset with the following hyperparameters :

$$\Theta^1_{\text{polyReg}} = \begin{cases} d \in \{1, 2, 3, 4\} \\ c \in \{0, 3, 7\} \end{cases} \tag{2}$$

TABLE 5 – Mean gridsearch results for Polynomial regression all region on Region - Hour dataset

d	c	Test Score	Train Score	Fit Time	Test Time
1	0	0.7418	0.8225	0.0614	0.0274
	3	0.9703	0.9814	0.0607	0.0276
	7	0.9715	0.9825	0.0650	0.0287
2	0	0.8434	0.9325	0.7397	0.0856
	3	0.9790	0.9911	0.8155	0.0933
	7	0.9807	0.9948	1.2358	0.1224
3	0	0.7784	0.9753	13.3555	0.3119
	3	0.9672	0.9961	31.6036	0.4749
	7	0.9693	0.9988	84.6625	0.7107

The results are even better at this resolution ; we can make the same conclusion : when the polynomial degree is 2, the results are constantly better. The small increase in the number of features reduces the risk of overfiting but when $d = 4$, most regions still overfit.
We also take a look at the coefficient distribution :

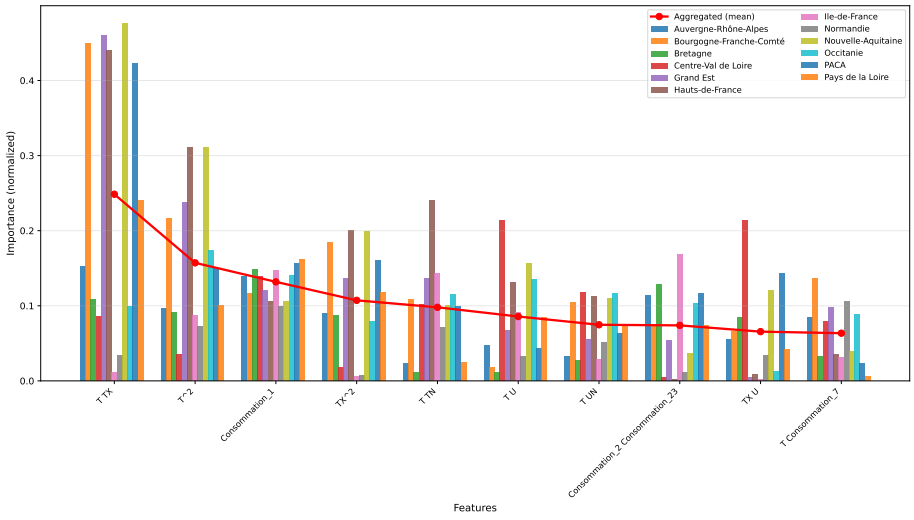


FIGURE 7 – Top hourly feature importance for polynomial regression

Surprisingly, the model does not depend as much as the previous one on the last consumption. It relies much more on the temperature. The coefficients seem to be more broadly distributed, and the weights vary a lot between different

regions ; it looks noisier than the daily feature ones.

4.2 Ridge

Ridge is extremely similar to polynomial regression but with a regularization term. We thought it might be interesting to test it, as we saw the previous models tended to overfit when the polynomial degree increased ; the regularization term should prevent such behavior.

Ridge regression introduces a new hyperparameter $\alpha \in \mathbb{R}^{+*}$ that controls the regularization strength. We had no idea on what type of value it should be set to, so we first went through a very broad grid search with 10 values from 10^{-4} to 10^{3} on a log scale. We also tried to optimize the polynomial degree and the number of last consumptions to consider.

4.2.1 Region - Day dataset

For the Region - Day dataset we tried those parameters to have a first sens of what the model is capabable.

$$\Theta^{1}_{\text{ridge}} = \begin{cases} d \in \{1, 2, 3, 4\} \\ \alpha \in \{10^{k} \mid k \in [-4, 3] \text{ with } n = 10\} \\ c \in \{0, 3, 7, 11\} \end{cases} \tag{3}$$

The table was quite huge, so we only show a section of it here. The results are the mean across all regions of the mean of cross-validation.

TABLE 6 – Mean gridsearch results for all region on the Region - Day dataset for Ridge

d	α	c	Test Score	Train Score	Fit Time	Test Time
2	0.0001	3	0.9748	0.9794	0.0397	0.0162
		7	0.9753	0.9818	0.0513	0.0152
		11	0.9745	0.9836	0.0715	0.0167
	0.774	3	0.9745	0.9790	0.0402	0.0155
		7	0.9752	0.9811	0.0544	0.0166
		11	0.9752	0.9825	0.0761	0.0165
	27.825	3	0.9715	0.9755	0.0404	0.0150
		7	0.9730	0.9780	0.0579	0.0155
		11	0.9742	0.9796	0.0753	0.0175
	1000	3	0.9539	0.9582	0.0397	0.0161
		7	0.9545	0.9611	0.0558	0.0166
		11	0.9571	0.9630	0.0747	0.0181
3	0.0001	3	0.9687	0.9861	0.1858	0.0220
		7	0.9443	0.9906	0.4851	0.0286
		11	0.7908	0.9951	1.3136	0.0348
	0.774	3	0.9753	0.9843	0.1829	0.0220
		7	0.9746	0.9863	0.4791	0.0296
		11	0.9723	0.9881	1.2906	0.0366
	27.825	3	0.9755	0.9821	0.1824	0.0228
		7	0.9760	0.9835	0.4802	0.0266
		11	0.9760	0.9844	1.2839	0.0379
	1000	3	0.9639	0.9719	0.1682	0.0218
		7	0.9679	0.9755	0.4234	0.0254
		11	0.9702	0.9775	1.2696	0.0310
4	0.0001	3	-7.5125	0.9947	1.8125	0.0423
		7	-19.3278	0.9992	4.9224	0.0834
		11	-5.2756	0.9999	9.6340	0.1604
	0.774	3	0.9299	0.9887	1.7496	0.0413
		7	0.9491	0.9914	4.4990	0.0773
		11	0.9430	0.9938	8.5723	0.1452
	27.825	3	0.9727	0.9850	1.7430	0.0407
		7	0.9721	0.9869	4.3617	0.0659
		11	0.9721	0.9885	8.4645	0.1252
	1000	3	0.9686	0.9780	1.8226	0.0434
		7	0.9727	0.9813	4.9303	0.0827
		11	0.9736	0.9827	9.7114	0.1624

As we can see, a really small alpha tends to overfit, especially as the polynomial degree increases, but overall all results are quite good, with most of

them above 0.97 in the test score. As expected, the regularization term did help with the third and the fourth polynomial degrees. On the fifth degree (not shown here), the overfit was constant, and even with a higher value of $\alpha \in [10,000, 50,000]$, the test score only decreased.

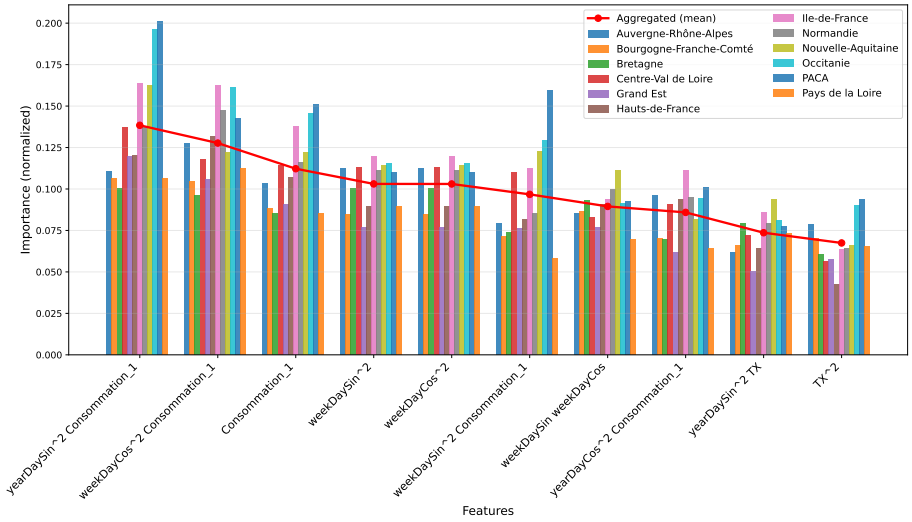


FIGURE 8 – Top daily feature importance for Ridge ($d = 3$, $c = 7$, $\alpha = 27$

This chart is quite impressive, as it shows the important weight of last consumption and the date, represented by *yearDay* and *weekDay* ; the temperature does not appear until the 8th coefficient. The other important remark is the consistency ; across all regions, a lot of features seem equally important. This can be partially explained by the increased number of features from augmenting the maximum polynomial degree ; we went from 630 features to 7770. This explanation can be confirmed by the next graphic that shows the same model but with $d = 2$.

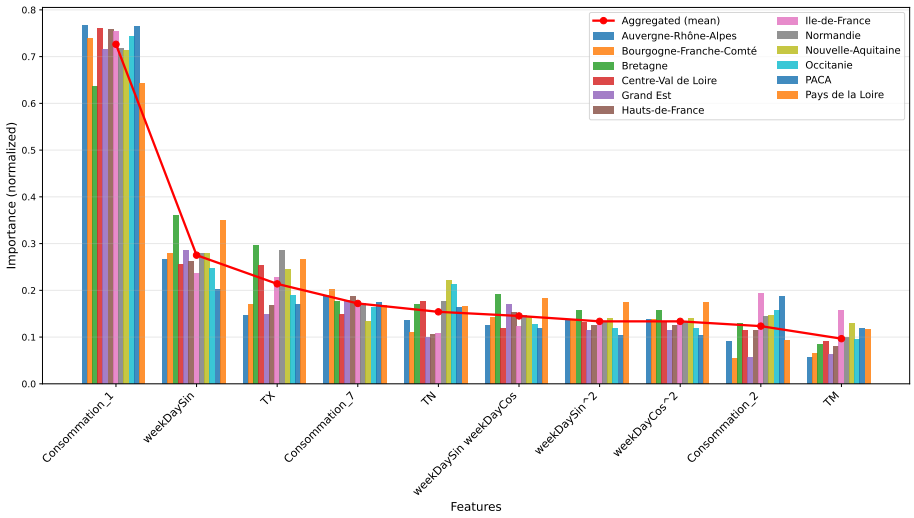


FIGURE 9 – Top daily feature importance for Ridge ($d = 2$, $c = 11$, $\alpha = 27$)

Indeed, in this graphic we can see the massive weight associated with the last consumption, meaning our model mainly uses this feature to predict its result, and it still yields a test score of 0.97. Another aspect we found on those two graphics is the tendency for all regions to use the same features ; the bars' tops are way closer than on the polynomial regression ones. This is the direct effect of the regression term, pushing the models towards the same important features.

We tried optimizing a bit further α with more values on a smaller scale, but it did not make any major improvement, as all test scores were around 0.97 ; we realize it only matters when there is a large gap between two α values.

We also tried extremes values, $\alpha \in \{10,000, 50,000, 100,000\}$ with a higher polynomial degree ($d \in \{4, 5, 6\}$) but the results were at best a bit lower than the one already mentioned, and most of the time they were negative as the model overfit.

4.2.2 Region - Hour dataset

On this dataset, we reduce the alpha scale, as we saw the extremely small value tends to reproduce the polynomial regression behavior. We also increased the number of last consumptions to consider, as it is a smaller scale.

$$\Theta^1_{\text{ridge}} = \begin{cases} d \in \{1, 2, 3, 4\} \\ \alpha \in \{10^k \mid k \in [-3, 3] \text{ with } n = 5\} \\ c \in \{0, 3, 7, 15, 24\} \end{cases} \tag{4}$$

TABLE 7 – Mean gridsearch results for Ridge all region on the Region - Hour dataset

d	α	c	Test Score	Train Score	Fit Time	Test Time
1	0.001	7	0.9645	0.9782	0.0584	0.0293
		24	0.9743	0.9878	0.0742	0.0330
	1	7	0.9647	0.9782	0.0668	0.0295
		24	0.9747	0.9877	0.0816	0.0334
	31.622	7	0.9661	0.9770	0.0639	0.0306
		24	0.9770	0.9861	0.0765	0.0361
2	0.001	7	0.9782	0.9929	0.5236	0.1158
		24	0.9854	0.9975	1.7114	0.2858
	1	7	0.9803	0.9911	0.5259	0.1119
		24	0.9871	0.9965	1.7448	0.2856
	31.622	7	0.9787	0.9881	0.5088	0.1232
		24	0.9857	0.9940	1.7298	0.2826
3	0.001	7	0.9544	0.9976	25.0545	0.9865
		24	0.9416	0.9996	1034.4073	6.1026
	1	7	0.9750	0.9961	24.2705	0.9838
		24	0.9860	0.9984	414.6369	5.7144
	31.622	7	0.9741	0.9939	24.8704	1.0113
		24	0.9818	0.9972	422.8574	6.4514

This table is only a short section of the whole table, but a few important pieces of information are emerging. The polynomial degree d does not matter as much as before for the test score, as we achieve a slightly better performance with it, $d = 2$ but the other degrees are quite good. But it is a heavy matter in the fit time, as it massively explodes for the third degree, especially when the number of features increases (when c is high). Computing this search grid took a full night and some parameter combinations could not be computed, as they created arrays too big for our computers' RAM space. We will address this problem later in the report with a PCA approach to reduce the number of features.

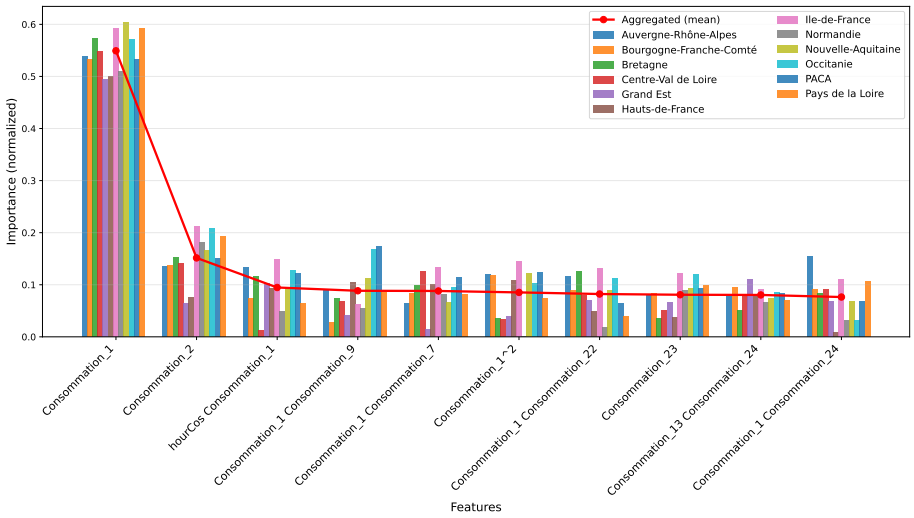


FIGURE 10 – Top hourly feature importance for Ridge ($d = 2$, $c = 24$, $\alpha = 1$)

We found this graphic extremely interesting, as it shows the strategy for the best result : only rely on the last consumptions. And especially on the first one, for all regions, more than half of their coefficient weight was on the last consumption recorded. It makes perfect sense because at this scale the consumption does not change a lot from an hour to the next one, giving a lot of information to our models.

4.3 Decision trees

We decided to test decision trees for our problem, as they are more general in function approximation, while we were limited to polynomial functions in the previous methods, and testing a lot of functions would have become unfeasible in a reasonable time.
The new hyperparameters to optimize are the maximum depth autorize m, the number of features to consider at each step f, we defined it using the default functions log2, sqrt, and None, which compute the value accepted depending on the total number of features. We continue to optimize the number of last consumption c.

4.3.1 Region - Day dataset

To get a sense of what our trees could do, we decide to start with a maximum depth of 3 and no restriction on the number of features used. We only show here the result for the region Auvergne-Rhône-Alpes in 2023. But all regions behave similarly.

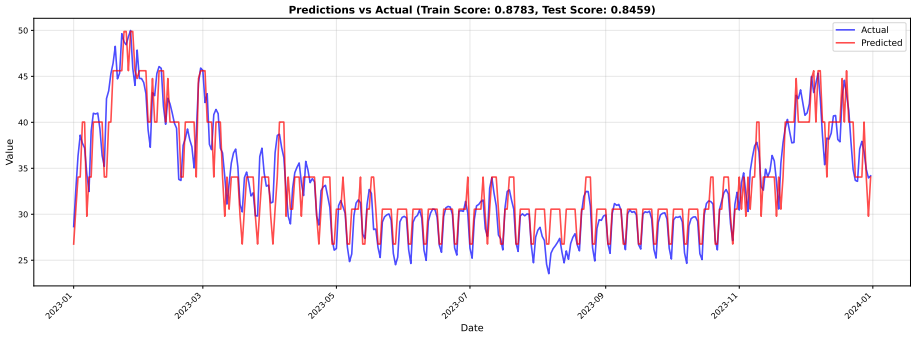


FIGURE 11 – Prediction for decTree on Auvergne-Rhône-Alpes Region - Day dataset for 2023 ($m = 3$, $f = None$, $c = 3$)

We can see that the results are promising with correct prediction, especially during the summer, but we can see that in July, when the consumption drops, the tree did not adapt ; we need to augment the maximum depth.
We decide to test those parameters to get a first sense of what seems important and how the parameters affect each other.

$$\Theta^1_{\text{decTree}} = \begin{cases} m \in \{5, 6, 7, 8, 9, 10, 11\} \\ f \in \{None, sqrt, log2\} \\ c \in \{0, 3, 7, 11\} \end{cases} \tag{5}$$

TABLE 8 – Mean gridsearch results for DecTree on all region on Region - Day

m	f	c	Test Score	Train Score	Fit Time	Test Time
5	None	3	0.9345	0.9533	0.0260	0.0122
		11	0.9267	0.9483	0.0359	0.0108
	log2	3	0.8744	0.9026	0.0161	0.0117
		11	0.8753	0.9128	0.0186	0.0114
	sqrt	3	0.8589	0.8933	0.0158	0.0112
		11	0.8665	0.9052	0.0165	0.0100
7	None	3	0.9506	0.9760	0.0379	0.0121
		11	0.9440	0.9748	0.0544	0.0117
	log2	3	0.9010	0.9455	0.0195	0.0114
		11	0.8950	0.9495	0.0230	0.0114
	sqrt	3	0.9079	0.9510	0.0211	0.0116
		11	0.9065	0.9514	0.0223	0.0130
9	None	3	0.9486	0.9884	0.0432	0.0121
		11	0.9414	0.9887	0.0675	0.0120
	log2	3	0.9154	0.9743	0.0211	0.0125
		11	0.9045	0.9761	0.0279	0.0120
	sqrt	3	0.9115	0.9745	0.0205	0.0115
		11	0.9017	0.9763	0.0276	0.0156
11	None	3	0.9439	0.9952	0.0481	0.0133
		11	0.9377	0.9955	0.0815	0.0123
	log2	3	0.9165	0.9890	0.0243	0.0116
		11	0.8968	0.9897	0.0289	0.0133
	sqrt	3	0.9082	0.9888	0.0242	0.0128
		11	0.8979	0.9903	0.0306	0.0109

As expected, increasing the maximum depth greatly helped in the first place, but then it started to overfit. Reducing the number of max features to log2 or sqrt only damaged the performance of the tree. The score is overall worse than the previous method, but the fit and test time are way better. For a quick estimation, this method seems perfect. We only show two different last consumptions since it did not have a big impact on the performance of our models.

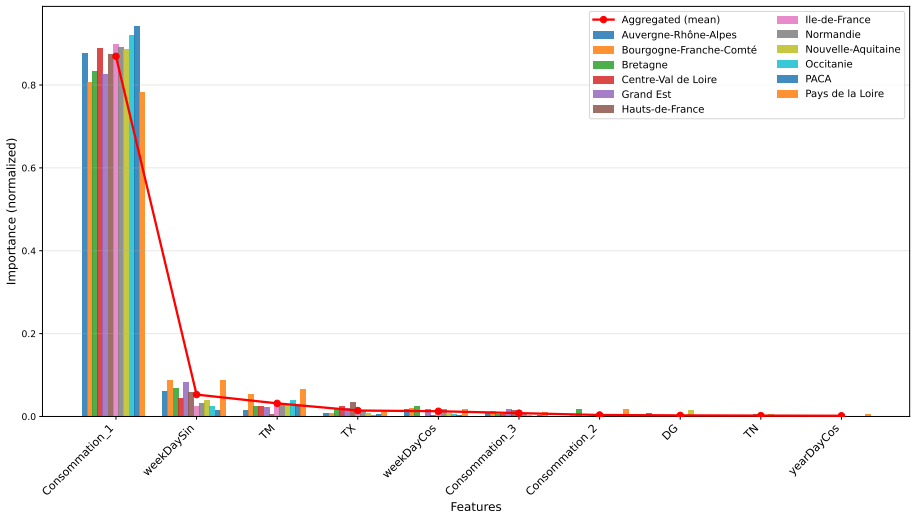


FIGURE 12 – Top daily feature importance for decTree ($m = 7$, $f = None$, $c = 3$)

Here the results are astonishing ; the models almost **only rely on the last consumption. All other features are ignored**. We tried different values for the minimum sample of leaves or the minimum sample for splitting ; the results are always the same with a massive dominance of the last consumption coefficients. Removing the last consumption feature led the model to rely on the temperature, but the scores massively dropped to around 0.86.

4.3.2 Region - Hour dataset

We decided to adapt our parameters by going a bit further on the number of maximum depths, as the hour dataset has more features in it.

$$\Theta^1_{\text{decTree}} = \begin{cases} m \in \{5, 9, 10, 11, 13, 15\} \\ f \in \{None, sqrt, log2\} \\ c \in \{0, 3, 7, 11, 24\} \end{cases} \tag{6}$$

TABLE 9 – Mean gridsearch results for decTree on all region on Region - Hour dataset

m	f	c	Test Score	Train Score	Fit Time	Test Time
5	None	0	0.6491	0.8364	0.1382	0.0221
		3	0.9547	0.9666	0.1660	0.0214
		24	0.9574	0.9690	0.3607	0.0273
	sqrt	0	0.5596	0.7108	0.0491	0.0183
		3	0.8057	0.8681	0.0534	0.0187
		24	0.8775	0.9066	0.0818	0.0230
11	None	0	0.7362	0.9713	0.2634	0.0209
		3	0.9777	0.9961	0.3939	0.0285
		24	0.9747	0.9956	0.8209	0.0267
	sqrt	0	0.6548	0.9251	0.0770	0.0198
		3	0.9072	0.9763	0.0965	0.0286
		24	0.9394	0.9842	0.1571	0.0335
15	None	0	0.7260	0.9947	0.3670	0.0225
		3	0.9768	0.9995	0.4921	0.0232
		24	0.9731	0.9994	0.9357	0.0266
	sqrt	0	0.6290	0.9817	0.1149	0.0245
		3	0.9137	0.9952	0.1292	0.0295
		24	0.9400	0.9974	0.1981	0.0336

On this dataset, the model performs significantly better as it equals the previous methods. Reducing the number of features definitely does not help in achieving a better score. We again showed the version without the last consumption, and we can see the massive gap created by it, while knowing the last 3 or 24 hours does not impact a lot.

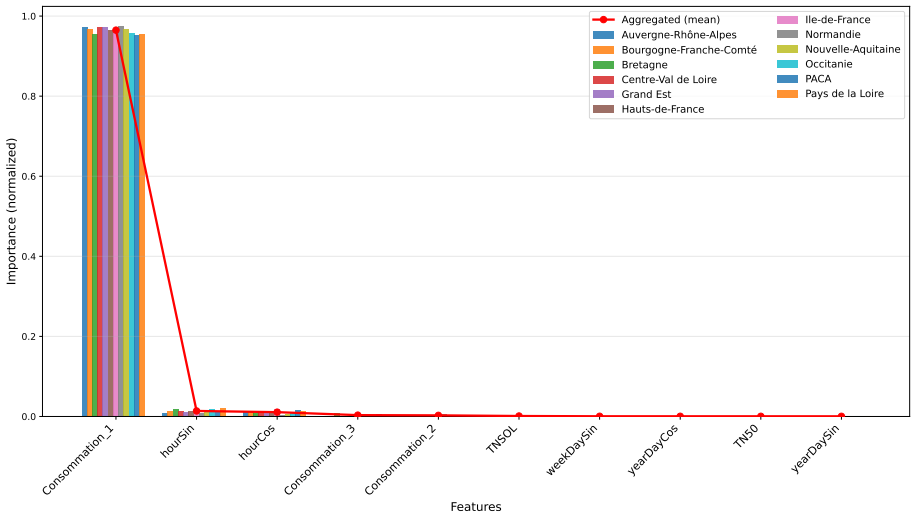


FIGURE 13 – Top hourly feature importance for decTree ($m = 11$, $f = None$, $c = 3$)

If it was impressive on the daily dataset, on this chart the last consumption dominance became absurd. We found two main explanations for this phenomenon. Since the decision tree chooses the features that most reduce the variance at each step, the last consumption has the two main ingredients, as it has itself a high variance and is extremely precise in determining the next consumption. Combining the two gives such a graph where the tree seems to only rely on one feature.

The results are quite good, but we seem to reach a ceiling as the train score are already above 0.99. To further improve the performance of our decision trees, we need a meta-estimator, AdaBoosting.

4.4 AdaBoosting our trees

AdaBoosting is a technique to optimize an estimator, in our case a decision tree. It starts by fitting the tree on our dataset, then it recursively creates copies of the tree and refits it on the data by focusing on the worst part.

This method introduces a new hyperparameter on top of the decision tree ones. The number of estimators to compute n.

Technical Note : The learning rate hyperparameter

AdaBoosting has another hyperparameter, the learning rate ; it decides how the weights are applied after each boosting iteration. In our testing we did not find any major change between different values, so we left it at 1, the default value for the rest of the project.

4.4.1 Region - Day dataset

We first went for a complete grid search with the following parameters :

$$\Theta^1_{\text{adaBoost}} = \begin{cases} m \in \{10, 11, 13\} \\ f \in \{None, sqrt, log2\} \\ c \in \{0, 3, 7, 9, 11, \} \\ n \in \{100, 200, 300\} \end{cases} \tag{7}$$

TABLE 10 – Mean gridsearch results for AdaBoost on all region on Region - Day dataset

m	n	Test Score	Train Score	Fit Time	Test Time
10	100	0.9717	0.9988	45.4596	0.0394
	200	0.9720	0.9989	102.1913	0.0647
	300	0.9721	0.9989	160.2984	0.0947
11	100	0.9718	0.9993	50.6038	0.0403
	200	0.9719	0.9994	115.7641	0.0664
	300	0.9719	0.9994	170.9372	0.0917
12	100	0.9713	0.9996	54.5517	0.0375
	200	0.9720	0.9996	125.9629	0.0667
	300	0.9721	0.9996	179.3253	0.0907
13	100	0.9715	0.9997	60.6802	0.0392
	200	0.9720	0.9998	120.2577	0.0670
	300	0.9719	0.9998	183.4907	0.0855

We only show the best portion of the table with $f = None$ and $c = 9$ as it was a very large table, the rest of it is not interesting for our point. The results were quite disappointing, as we did not make any major progress despite fitting time rapidly increasing. In all cases we end up with a score around 0.97 without being able to go beyond. The AdaBoost pushes every parameter combination to the ceiling without breaking it. A way to improve our performance would be to keep increasing the number of estimators, but this quickly becomes computer-intensive.

As we saw previously, the number of last consumptions had no big impact as long as there was one, so we froze this parameter to $c = 3$. We also saw that reducing the number of maximum features only led to a reduced test score, so we also fixed $f = None$, we also fixed $m = 12$ as it seemed the most promising parameter. Those changes should help us keep the computer time reasonable as we explore the AdaBoost technique, which requires a lot of computer resources. We decide to launch a new grid search focusing on the number of estimators :

$$\Theta^2_{\text{adaBoost}} = \begin{cases} m \in \{12\} \\ f \in \{None\} \\ c \in \{3\} \\ n \in \{600, 1000\} \end{cases} \tag{8}$$

TABLE 11 – Mean gridsearch results for AdaBoost on all region on Region - Day dataset

m	n	Test Score	Train Score	Fit Time	Test Time
12	600	0.9726	0.9996	21.3192	0.1508
	1000	0.9725	0.9996	34.0807	0.2276

The dataset is maxed out, as we did not make any improvement despite greatly increasing the number of estimators.

4.4.2 Region - Hour dataset

TABLE 12 – Mean gridsearch results for AdaBoost on all region on the Region - Hour dataset

		region				Moyenne
			Test Score	Train Score	Fit Time	Test Time
m	n	c				
7	100	3	0.9820	0.9936	16.2888	0.1809
		24	0.9841	0.9941	33.9755	0.2797
	200	3	0.9823	0.9938	29.4590	0.3406
		24	0.9844	0.9944	61.4041	0.5393
	300	3	0.9823	0.9939	41.4619	0.4985
		24	0.9844	0.9944	88.7431	0.8100
10	100	3	0.9850	0.9984	24.7403	0.2166
		24	0.9865	0.9985	54.3522	0.3233
	200	3	0.9851	0.9985	47.7007	0.4219
		24	0.9868	0.9986	104.0279	0.6261
	300	3	0.9851	0.9986	69.9924	0.6239
		24	0.9868	0.9986	153.0583	0.9264
13	100	3	0.9851	0.9997	33.8977	0.2618
		24	0.9867	0.9998	70.8513	0.3741
	200	3	0.9853	0.9998	66.4715	0.5147
		24	0.9868	0.9998	140.6886	0.7381
	300	3	0.9853	0.9998	98.0066	0.6971
		24	0.9868	0.9998	198.7064	0.9848

The conclusion is the same as the daily dataset ; now all parameter combinations are viable, but there is no major breakthrough. Moreover, it seems like we maximized the training data with a train score at 0.999. And we already saw in the daily dataset that increasing the number of estimators will not lead to better results. We may need more information in our dataset to go beyond that point.

4.5 First summary

TABLE 13 – Best performance summary

model	delta	Test Score	Train Score	Fit Time	Test Time
AdaBoost	ByDay	0.9721	0.9996	16.3253	0.0907
	ByHour	0.9868	0.9998	198.7064	0.9848
Dec Tree	ByDay	0.9506	0.9760	0.0379	0.0121
	ByHour	0.9777	0.9961	0.3939	0.0285
Poly Reg	ByDay	0.9753	0.9818	0.1867	0.0382
	ByHour	0.9807	0.9948	1.2358	0.1224
Ridge	ByDay	0.9760	0.9835	0.4802	0.0266
	ByHour	0.9871	0.9965	1.7448	0.2856

This table summarizes the best test score obtained for each model on the different datasets. The other statistics are the best model's ones. We can read it like this : "For the Ridge on the Region - Hour dataset, the best mean score across all regions was 0.9871 and it was obtained with a train score of 0.9965 and 1.7448 seconds of fit time."

We were already quite impressed with the result we obtained in our different methods, but we can clearly see that something is missing. Our models are already quite performant, but remember that the only weather data they have access to is the mean for the whole region. This choice, made for the sake of simplicity and computer efficiency, may be the problem for breaking the 0.97 ceiling on the Region - Day datasets.

5 New datasets ?

5.1 Creating the Departments dataset

Instead of computing the mean on the whole region, we will compute it department-wise.
In some rare cases, no information was recorded on the whole department ; in those cases, we took the mean region-wise. Then we add to our dataset all departments' means, followed by "_X" where X is the department number. We can do this operation at a daily and hourly scale. We thus have two brand new classes of datasets :

1. Dep - Day datasets
2. Dep - Hour datasets

We face an immediate problem with this approach : the number of features has been multiplied. Some regions have more than 70 features on the Dep - Day

dataset, and in the Dep - Hour this number goes up to 158. Hopefully those features are redundant, and we should be able to reduce them easily.

5.2 Introduction the PCA

The principal component analysis (PCA) helps us reduce the number of dimensions, in our case, the number of features, while keeping as much information as possible. We need to determine how much the information will be compressed. To help us decide, we plotted the information loss in comparison with the number of components used. This approach gives us the following plot :

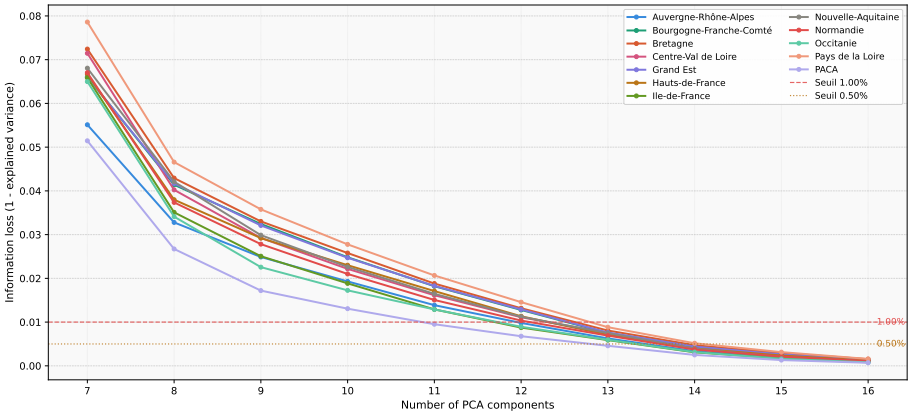


FIGURE 14 – Information loss by the number of PCA components, Region - Day

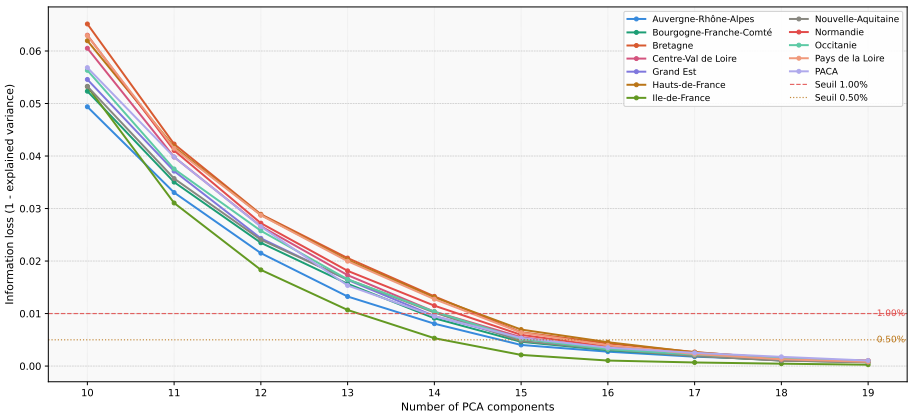


FIGURE 15 – Information loss by the number of PCA components, Region - Hour

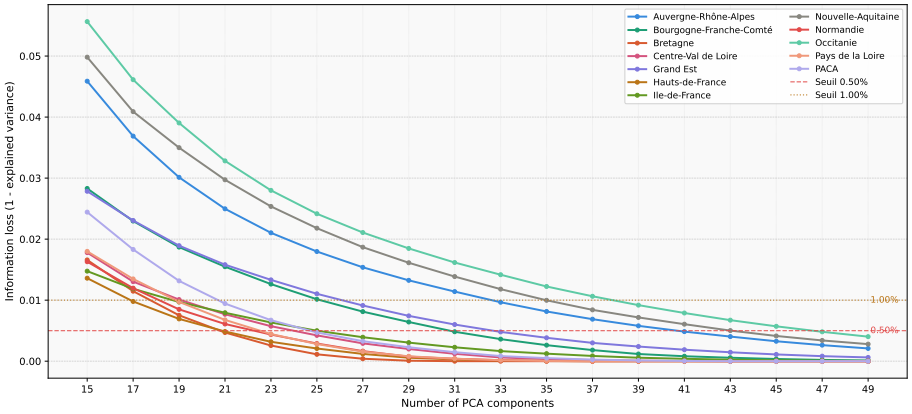


FIGURE 16 – Information loss by the number of PCA components, Dep - Day

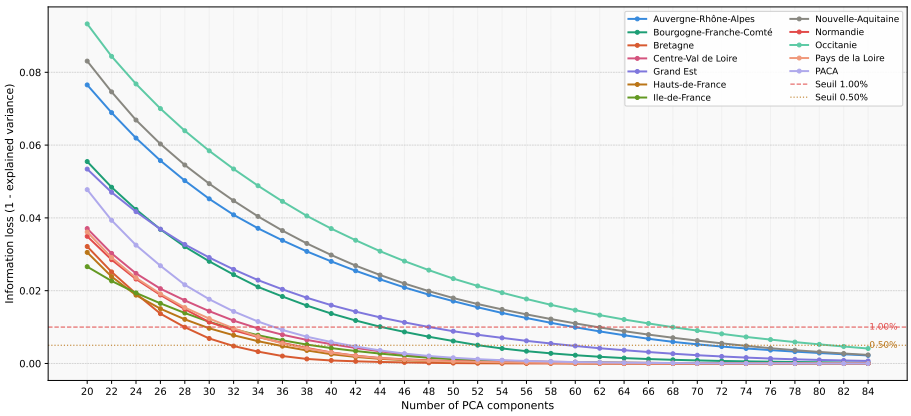


FIGURE 17 – Information loss by the number of PCA components, Dep - Hour

We can see that in all cases, the number of features can indeed be drastically reduced while keeping more than 99% or even 99.5% of the information. Since all regions behave differently, we wanted something more general than fixing a set parameter for our PCA, so we divided the precision into three categories : losing 0.01, 0.005 or 0 information. 0 means no PCA was introduced.
We then create a little function helper that, from this value, finds the number of components (nComps) to include in the PCA depending on the region and the dataset.
In our grid search we now have a new hyperparameter, the PCA precision, $p \in \{0.01, 0.005, 0\}$.

Table 14 – Feature reduction by precision in all datasets

Region	Precision	Region - Day nComps	Region - Hour nComps	Dep - Day nComps	Dep - Hour nComps
Auvergne-Rhône-Alpes	0	14	26	79	155
	0.005	11	15	41	72
	0.01	10	14	33	60
Bourgogne-Franche-Comté	0	14	26	55	107
	0.005	11	15	31	54
	0.01	10	14	27	46
Bretagne	0	14	26	31	59
	0.005	11	16	21	32
	0.01	10	15	19	28
Centre-Val de Loire	0	14	26	43	83
	0.005	11	16	25	42
	0.01	10	15	21	34
Grand Est	0	14	26	67	131
	0.005	11	16	33	60
	0.01	10	14	27	50
Hauts-de-France	0	14	26	37	71
	0.005	11	16	21	36
	0.01	10	15	17	30
Ile-de-France	0	14	26	55	107
	0.005	11	15	25	40
	0.01	10	14	19	32
Normandie	0	14	26	79	155
	0.005	11	16	23	38
	0.01	10	15	19	32
Nouvelle-Aquitaine	0	14	26	37	71
	0.005	11	15	45	74
	0.01	10	15	35	62
Occitanie	0	14	26	85	167
	0.005	11	16	47	82
	0.01	10	15	39	68
PACA	0	14	26	37	71
	0.005	10	16	25	42
	0.01	9	14	21	36
Pays de la Loire	0	14	26	43	83
	0.005	11	16	23	38
	0.01	11	15	19	32

6 Going back to the region datasets

Before seeing the new datasets in action, we wanted to see the possible time gain offered by PCA on the previous models. So we recompute all grid searches previously mentioned but with a PCA precision to 0.005 and 0.01.

TABLE 15 – PCA Impact on Model Performances on the dataset at Region scale

model	delta	p	testScore	trainScore	fitTime	testTime
AdaBoost	Day	0	0.9721	0.9996	16.3253	0.0907
		0.005	0.8699	0.9645	4.7579	0.0858
	Hour	0	0.9868	0.9998	198.7064	0.9848
		0.005	0.8002	0.9650	49.4257	0.5346
		0.01	0.6579	0.7590	7.5432	0.1672
Dec Tree	Day	0	0.9506	0.9760	0.0379	0.0121
		0.005	0.8501	0.8956	0.0226	0.0127
	Hour	0	0.9777	0.9961	0.3939	0.0285
		0.005	0.8238	0.8803	0.2759	0.0323
		0.01	0.7131	0.8224	0.0960	0.0255
Poly Reg	Day	0	0.9753	0.9818	0.1867	0.0382
		0.005	0.9732	0.9766	0.0519	0.0182
	Hour	0	0.9807	0.9948	1.2358	0.1224
		0.005	0.8671	0.9346	2.0234	0.2048
		0.01	0.7622	0.8829	0.0910	0.0362
Ridge	Day	0	0.9760	0.9835	0.4802	0.0266
		0.005	0.8887	0.8966	0.0294	0.0136
	Hour	0	0.9871	0.9965	1.7448	0.2856
		0.005	0.8267	0.8999	0.5665	0.1202
		0.01	0.7882	0.8723	0.1215	0.0444

This table shows the best model for each PCA precision in all configurations. The results are the mean across all regions ; we only look at the best model in regard to the test score.

We were really disappointed when we saw those results, as the PCA massively reduced the best score possible. On the other hand, it greatly reduces the computation required, and the train score is lower too, possibly meaning that we have room for improvement.

7 Testing our models on the new datasets

As we already introduced the used model, we found it more interesting to compare their performance on the same datasets. To see what methods perform better on those datasets.

7.1 The Daily dataset

7.1.1 Polynomial regression

TABLE 16 – Mean gridsearch results for Polynomial regression on all region on the Dep - Day dataset

d	c	p	Test Score	Train Score	Fit Time	Test Time
1	0	0	0.8086	0.8685	0.0559	0.0177
		0.001	0.7034	0.7324	0.0421	0.0184
		0.01	0.5202	0.5643	0.0560	0.0166
	3	0	0.9486	0.9529	0.0541	0.0155
		0.001	0.7136	0.7407	0.0399	0.0168
		0.01	0.5248	0.5705	0.0454	0.0164
2	0	0	-66.8669	0.9708	11.4835	0.1397
		0.001	0.7770	0.8254	0.0728	0.0226
		0.01	0.5457	0.6348	0.0467	0.0187
	3	0	-1147.7299	0.9916	13.6292	0.1525
		0.001	0.7916	0.8345	0.0927	0.0205
		0.01	0.5448	0.6455	0.0432	0.0185
3	0	0	-52.5161	1.0000	167.7092	0.3528
		0.001	-467123.0305	0.8528	0.6439	0.0239
		0.01	-0.2257	0.6852	0.0973	0.0280
	3	0	-2.5186	1.0000	187.9294	0.3807
		0.001	-271743.1253	0.8621	0.5764	0.0252
		0.01	-0.2744	0.7010	0.0968	0.0187

Those results are not surprising, as the number of features augments the chance of overfitting as well. Since the number of features explodes as the polynomial degree increases, we tend to overfit even for $d = 2$. Not all regions are overfitting for the second polynomial degree, but most of them are.

7.1.2 Ridge

TABLE 17 – Mean gridsearch results for Ridge on all region on Dep - Day datasets

d	α	p	Test Score	Train Score	Fit Time	Test Time
2	0.001	0	0.471	0.991	1.385	0.037
		0.001	0.792	0.835	0.046	0.018
		0.01	0.545	0.645	0.039	0.017
	0.032	0	0.904	0.990	1.384	0.038
		0.001	0.792	0.835	0.045	0.016
		0.01	0.545	0.645	0.035	0.015
	1	0	0.962	0.987	1.339	0.045
		0.001	0.793	0.834	0.044	0.018
		0.01	0.545	0.645	0.037	0.016
	31.623	0	0.970	0.981	1.372	0.047
		0.001	0.793	0.831	0.043	0.016
		0.01	0.548	0.643	0.035	0.016
	1000	0	0.959	0.968	1.477	0.056
		0.001	0.718	0.748	0.045	0.016
		0.01	0.523	0.568	0.035	0.015
3	0.001	0	-1.567	1.000	351.410	0.761
		0.001	-733.533	0.859	0.184	0.019
		0.01	-0.274	0.701	0.055	0.017
	0.032	0	0.344	0.999	361.084	1.997
		0.001	-34.434	0.856	0.154	0.019
		0.01	-0.256	0.701	0.056	0.017
	1	0	0.790	0.997	204.472	0.495
		0.001	-1.897	0.853	0.143	0.018
		0.01	0.039	0.700	0.055	0.017
	31.623	0	0.940	0.992	295.582	0.536
		0.001	0.658	0.845	0.138	0.018
		0.01	0.523	0.683	0.051	0.016
	1000	0	0.962	0.984	81.941	0.442
		0.001	0.692	0.762	0.144	0.018
		0.01	0.517	0.580	0.052	0.017

Here we can see the importance of the regularization term, while in the polynomial version, the models would overfit they stay under control here. With a α strong enoug even the third degree is possible, but we start to lose precision.

What is surprising is that it is worse than the Region - Day dataset, while it should be better with more precise information. Let's look at its coefficient to understand what is happening.

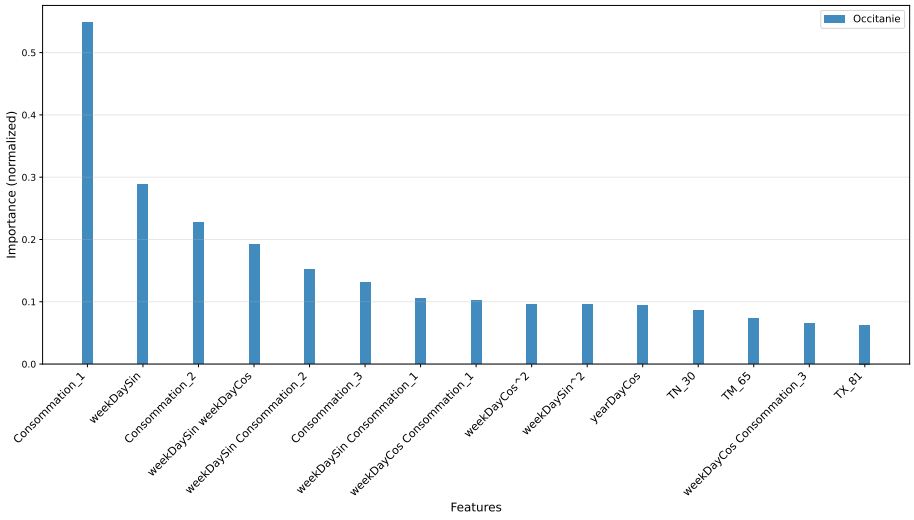


FIGURE 18 – Feature importances for Ridge on Dep - Day datasets, Occitanie

We cannot show the aggregate version, as all regions have different features. So here is the Occitanie one ; on all graphs, we see a clear dominance of the last consumption and, for the 10 first coefficients, no weather features. This is normal, as they are now each impacting at a smaller scale, thus having smaller coefficients.

7.1.3 DecTree

TABLE 18 – Mean gridsearch results for Dec Tree on all region on Dep - Day datasets

m	f	c	p	Test Score	Train Score	Fit Time	Test Time
5	None	9	0	0.9305	0.9554	0.1259	0.0150
			0.005	0.8826	0.9219	0.0904	0.0163
			0.01	0.8827	0.9219	0.0789	0.0142
11	None	9	0	0.9350	0.9960	0.2159	0.0141
			0.005	0.8782	0.9923	0.1368	0.0158
			0.01	0.8768	0.9920	0.1262	0.0138
13	None	9	0	0.9320	0.9986	0.2476	0.0150
			0.005	0.8718	0.9972	0.1595	0.0136
			0.01	0.8758	0.9971	0.1393	0.0141
15	None	9	0	0.9299	0.9995	0.2439	0.0136
			0.005	0.8733	0.9991	0.1615	0.0128
			0.01	0.8731	0.9991	0.1442	0.0143

The result are quite good, but compare to the other methods they are quite bad, especially the PCA massively reduce the performance of our models.

7.1.4 AdaBoost

TABLE 19 – Mean gridsearch results for AdaBoost on all region on Dep - Day datasets

m	n	c	p	Test Score	Train Score	Fit Time	Test Time
7	200	9	0.005	0.9461	0.9852	11.6741	0.0615
			0.01	0.9448	0.9844	9.5007	0.0579
	300	9	0.005	0.9464	0.9854	16.8572	0.0810
			0.01	0.9458	0.9846	13.8134	0.0772
10	200	9	0.005	0.9430	0.9980	16.2945	0.0610
			0.01	0.9434	0.9978	13.5268	0.0620
	300	9	0.005	0.9439	0.9980	24.2425	0.0881
			0.01	0.9439	0.9979	20.0035	0.0923
13	200	9	0.005	0.9431	0.9997	18.5115	0.0678
			0.01	0.9426	0.9996	15.2872	0.0690
	300	9	0.005	0.9433	0.9997	25.2421	0.0737
			0.01	0.9430	0.9996	22.6490	0.0893

The results are quite bad ; we could not compute the non-PCA version as the time to compute exploded. The same effect as the last time : we hit a ceiling at 0.95, and AdaBoosting did not break it, but all parameter combinations are optimized to that ceiling.

7.2 Hourly Datasets

On those datasets, since the number of features and rows was much larger, our computer could not process the non-PCA version. And the result where not interesting since improvement has been made. Here are the best results obtained by the different models. We only look at the mean of all region, as they most of the time behave similarly.

1. Polynomial regression : 0.9718 (with $d = 2$, $c = 24$, $p = 0.005$)
2. Ridge : 0.9771 (with $d = 2$, $\alpha = 100$, $c = 24$, $p = 0.005$)
3. Dec Tree : 0.7908 (with $m = 11$, $f = None$, $c = 24$, $p = 0.01$)
4. AdaBoost : 0.8632 (with $m = 10$, $n = 300$, $c = 5$, $p = 0.001$)

As we can see, the tree performs better on a low PCA, while polynomial and ridge are better the more information they have. But overall the results are not better than the ones obtained with the region datasets and often without PCA.

8 Filling the holes

8.1 Interpolate the missing values

We mentioned at the beginning that a certain amount of values were missing, and we ignored them when computing the mean. But we wanted to try to interpolate the missing values to see if the performance of our models could be even better.

8.2 First idea

Let's say we have a missing value v_t at time t, our first idea was to compute the following :

$$v_t = \frac{v_{t-1} + v_{t+1}}{2} \tag{9}$$

But when looking closer at our weather data, we realize that the missing values are often grouped up, like if there has been no record for a week or a month. So this idea felt short.

8.3 Nearest Neighbors

So we went back to look at all the datasets provided by *Météo France* and we saw that we had all the weather stations coordinated. We thus decide to take the mean of the 5 closest stations to interpolate our missing values.

This approach seems simple at first but was, in fact, much tougher.
To efficiently compute the distance between the stations and find the closest ones, we first create a file with all station information, then use a **BallTree** from sklearn with a haversine metric. We wanted to compute the closest stations in France, not just the department, but our computer could not load all the data at the same time. So we had to load only the adjacent departments.
The main algorithm went as follows :

Loop on all departments,
Loop on all of their stations,
Fill the missing values column by column.

A **special case** occurs when none of the 5 nearest stations had v_t, in this case, we decided to extend the search until we found one station with the required value.

We first did it for the daily weather datasets, and it took around 30 hours to compute. Since the hourly datasets are around 10 times bigger, we decided it was unreasonable to fill them.

We then create the full dataset by adding the electricity consumption data and computing the mean like described in the region and departments scale datasets. So we now have **two new classes of datasets to test our models on : Region - Day - fill and Dep - Day - fill.**

9 The filled datasets's results

To test our models on those new datasets, we again implemented the same technique with grid search ; we won't go into much detail on this part as it is quite redundant with the work already presented. Instead, we will only show the best score obtained by our different techniques.

Model	Scale	Delta	Test Score	Train Score	Fit Time	Test Time
AdaBoost	Dep	Daily	0.9732	0.9993	35.2996	0.0796
	Region	Day	0.9738	0.9993	8.2458	0.0565
Dec Tree	Dep	Daily	0.9378	0.9972	0.2509	0.0151
	Region	Daily	0.9393	0.9949	0.0783	0.0166
Poly Reg	Dep	Daily	0.9568	0.9666	0.2734	0.0197
	Region	Daily	0.9755	0.9808	0.0877	0.0129
Ridge	Dep	Daily	0.9713	0.9808	0.7475	0.0306
	Region	Daily	0.9764	0.9825	0.3192	0.0237

As we can see, there has been no major improvement with the new datasets. The interpolation did give a lot of information to our models. Ridge and Adaboost are clearly better, but among those two, Ridge is much faster.

10 Final summary

In this project we went across multiple models on different datasets, but at the end, what is the best to precisely predict the next day or hour's electricity consumption ? We organize a competition where only the best of the best are selected. To find our final winner, we will **compare models on data they have never seen before, the year 2024**.
To select the challengers, for each model, we took the best-performing model parameters found in our cross-validation for each region. Then we train the contender on the whole training set and then test them on 2024.

10.1 For the Day prediction

10.1.1 Polynomial regression

Our first contender is the polynomial regression ; for all regions, the best combination was the Region - Day unfilled datasets with the polynomial degree at 2 and no PCA. Here is a table showing the performance for each region.

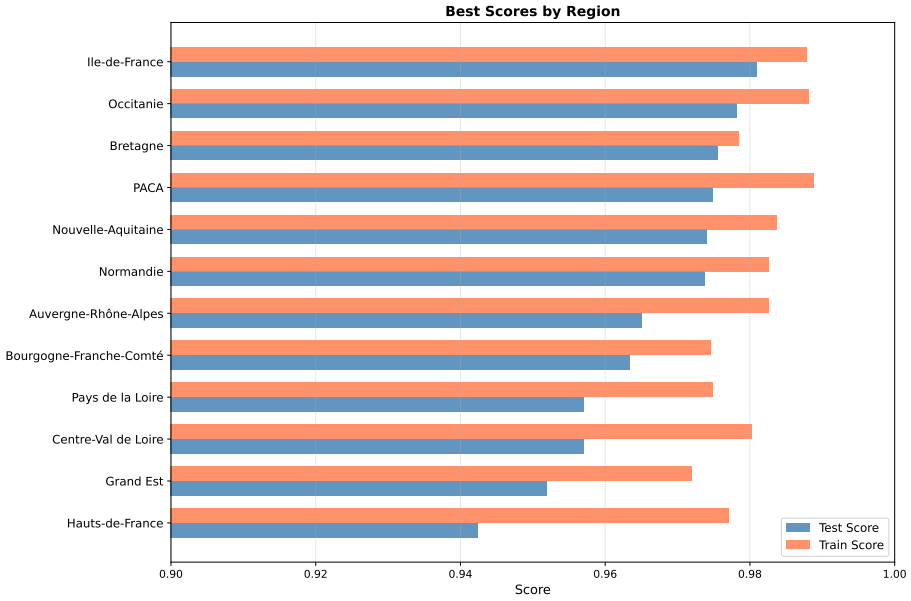


FIGURE 19 – Best score for the polynomial regression for daily datasets

10.1.2 Ridge

For the ridge regression, all regions went for the third polynomial degree with no PCA, but we have a bit of variety in the α choice and the number of last consumption to consider. Overall the results are correct but not as good as expected.

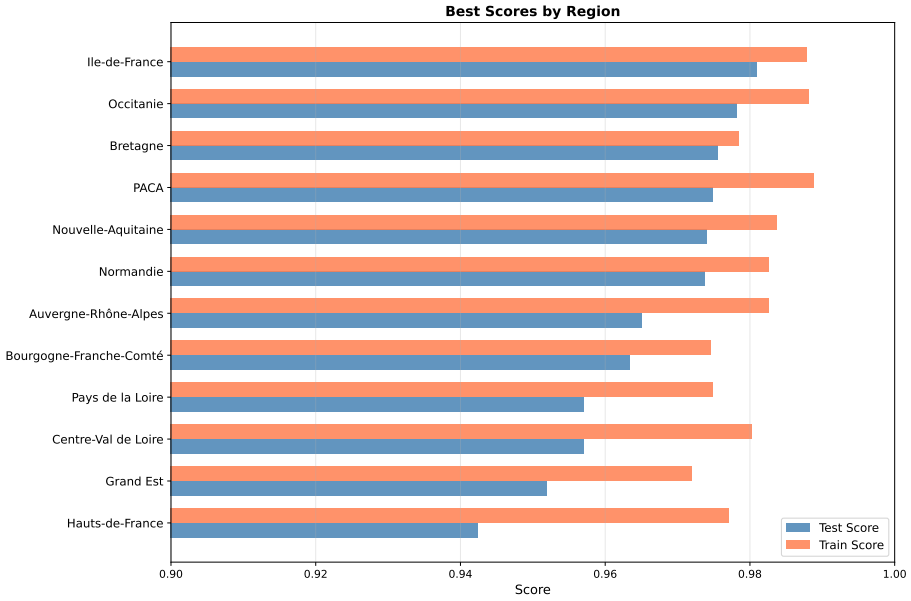


FIGURE 20 – Best score for Ridge on the daily datasets

10.1.3 Decision Trees

Our third contender did not shine previously, but we still gave him a chance. Without any suspense, the results are not great, but they are still honorable, with most of the region clearing the 0.90 bar. For all regions, the best option was the Region datasets without any PCA, no restriction on the number of features, and a max depth of 7 or 8.

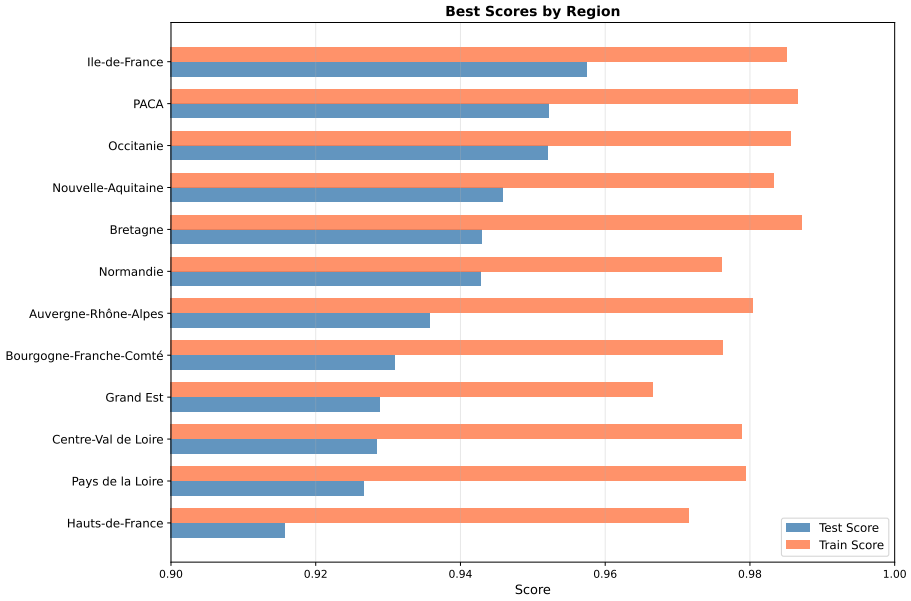


FIGURE 21 – Best score for Decisions Trees on the daily datasets

10.1.4 Adaboosting

Here we meet a challenger for the title ; this is the only one that performs better on the departments' datasets, without a PCA transformation and often with 300 estimators, sometimes only 200. But the max depth varies quite a lot from 13, 7, or 10. But overall the results were not what we expected ; it performed worse than the simple polynomial regression. It looks like it may have slightly overfit with an excellent score on the train set.

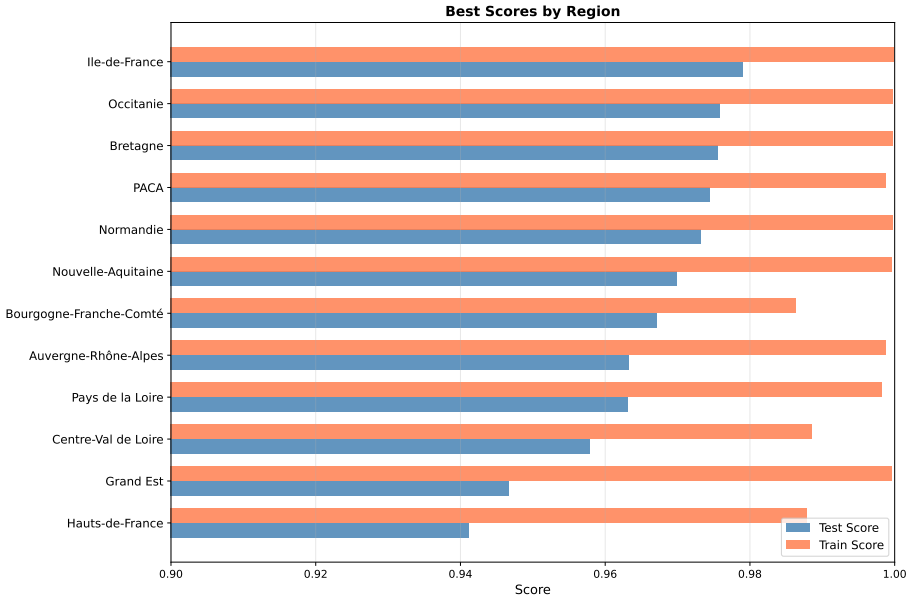


FIGURE 22 – Best score for AdaBoost on the daily datasets

10.1.5 Summing up the daily dataset

Our winner is clear ; the ridge regression outperforms every other model by a margin. But after all those figures, we can also see that regions are tougher than others ; for example, l'Île-de-France has been well understood by the models, while Haut-de-France looks harder to predict.

10.2 Hourly dataset

Will the ridge regression also win in the hour category ? The dataset is 10 times larger for this scope, while only containing 5 years, four to train and the last one to test. The difficult part for our challenger is to not overfit or focus on some details, forgetting the big picture. But knowing the last hour's consumption should help them make accurate bets on the future, let's find out who is our winner.

10.2.1 Polynomial regression

As for the daily datasets, all models perform better at the region scale, without any PCA and a polynomial degree of 2. The results are already really good, with all regions almost crossing the 0.97 test score. The most surprising part is the generalization of the model, as it performs equally or even better on the test dataset than the training dataset.

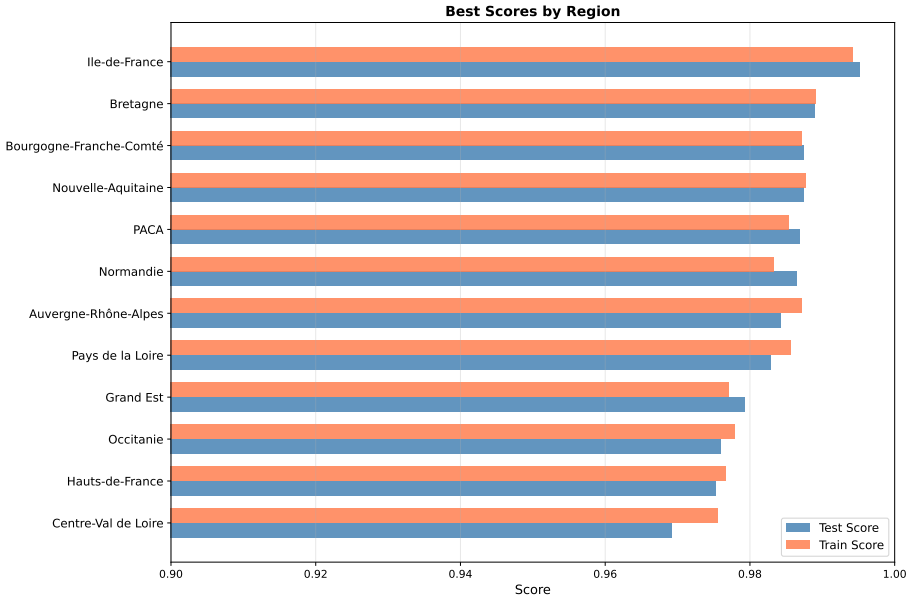


FIGURE 23 – Best score for polynomial regression on the hourly datasets

10.2.2 Ridge regression

Without more suspense, the results were amazing, with more than 0.98 test scores in all regions ; it raised the bar extremely high. With 5 regions out of the 12 with a test score above 0.99, this model shows its full capacity. For all regions, the best combination was the region data set without any PCA transformation, a polynomial degree of 2 except for Nouvelle-Aquitaine and Occitanie with 3, and a α ranging from 0.03 to 1000 for the third-degree models.

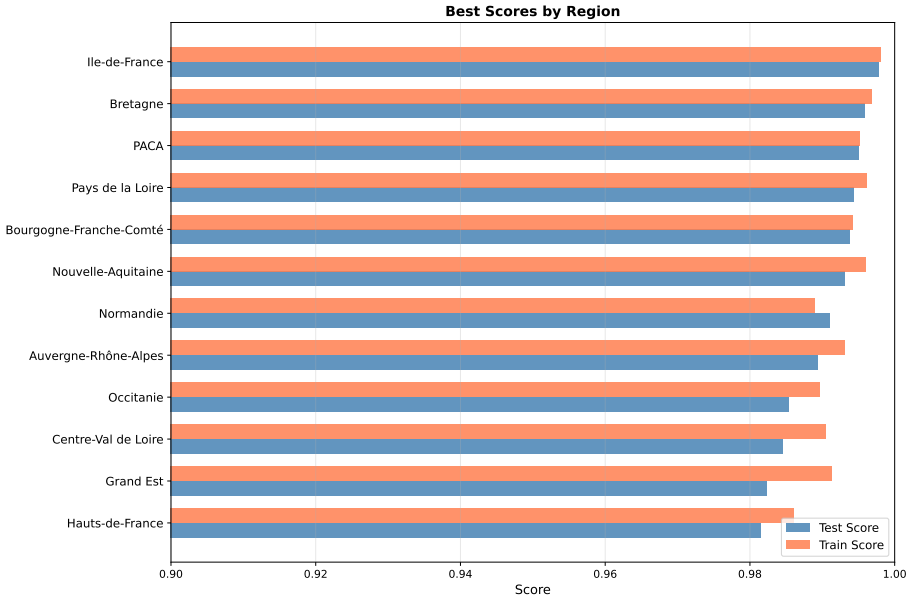


FIGURE 24 – Best score for ridge regression on the hourly datasets

10.2.3 Decision Trees

The decision trees are not giving up ; the smallest scale can be a great advantage for them as long as they don't overfit on the training dataset.
The results are not as good as the ridge model, but they still all clear the 0.98 mark. The train scores are not too high ; they did not fall into the overfitting trap as we might expect.

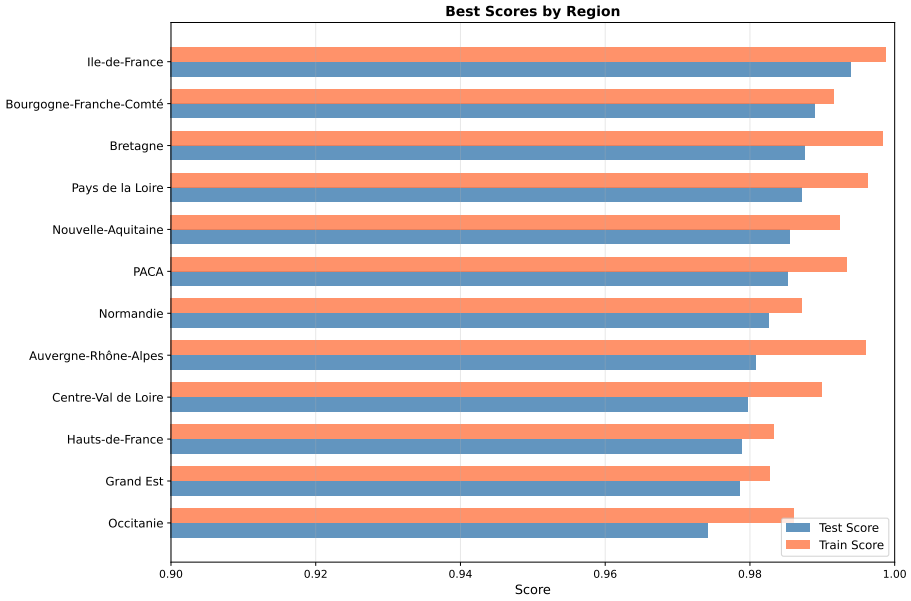


FIGURE 25 – Best score for ridge regression on the hourly datasets

10.2.4 AdaBoost

They were quite disappointing on the daily datasets ; maybe a smaller scale will please them more ?
The results are quite amazing, all the region are above 0.98 and Île-de-France reach 0.997 test score. This time, all models perform better at a Region scale except for PACA's one, they all needed 300 estimators and only PACA and Pays de la Loire used a PCA, the first we saw the PCA transformation used in the quest for the best score.

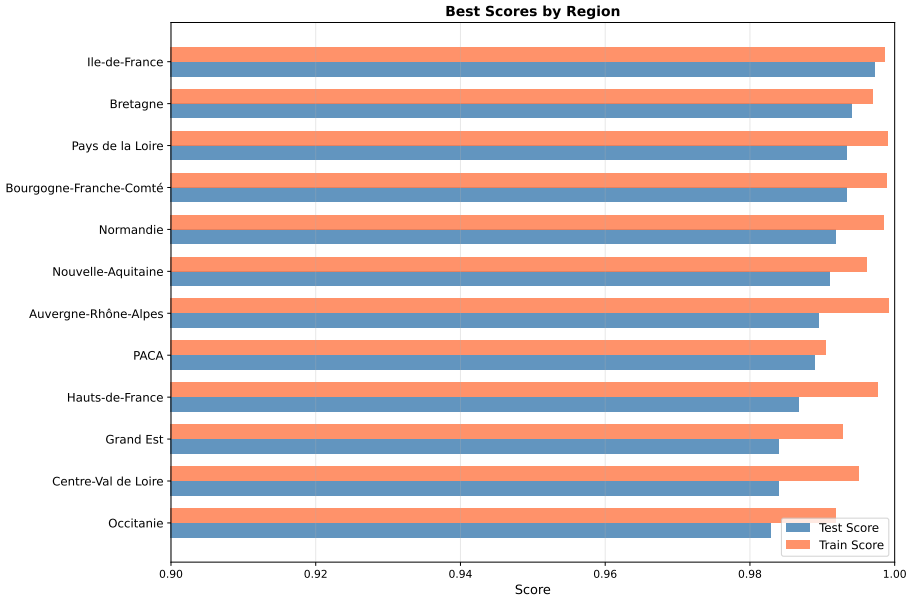


FIGURE 26 – Best score for adaBoost on the hourly datasets

10.2.5 Summary of the hourly dataset

In this case the battle is much closer, but the Ridge models did compute around 10 times faster, for approximately the same result as the adaboosting decision trees.

11 Conclusion

In this project we explore multiple machine learning techniques to predict French electricity consumption. We also had to deal with missing values and try different approaches to compute the datasets while balancing between precision and computer efficiency. We finally found that Ridge regression was the best model to estimate the electricity consumption, even though it relied much more on the previous consumption and date than weather information.

11.1 Comparing against RTE prediction

RTE france make their own prediction on the evolution of the electricity consumption ; unfortunately, they only provide us with insight at the national scale. Nevertheless, we found it interesting to compare ourselves to the official prediction and a more elaborate algorithm.

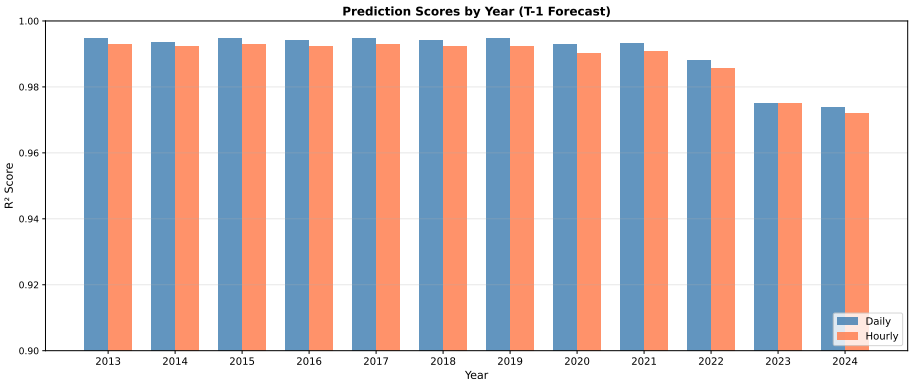


FIGURE 27 – Prediction score of RTE on a national scale

Their predictions were extremely accurate a few years ago, but we did not find any reason why their accuracy dropped in the last two years. We can proudly say that our models did match the recent *RTE* prediction accuracy at least in the recent years.

11.2 What's next

We end our research, but we think there is still room for improvement ; here is a list of our ideas that could be explored.

11.2.1 Weather station weights

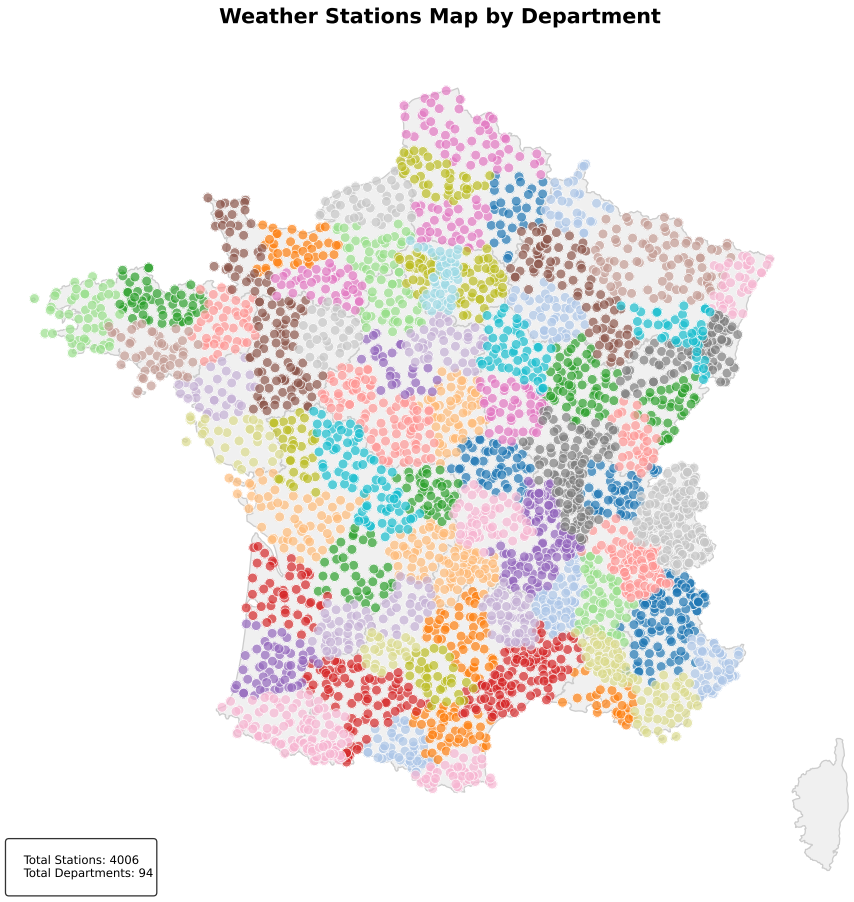


FIGURE 28 – Weather stations map, colored by departements

As we can see on this map, some spaces are heavily crowded with weather stations while other are more sparse, resulting in misleading means, as a weather change in a small area could be registered by multiple weather stations.

When computing the dataset, instead of taking the simple mean over all stations, add weights to each station. Those weights could be calculated in various ways.

Area perimeter An idea would be to compute the area covered by each station ; if a station is isolated, it would have more weights than 3 close-by stations.

Population perimeter A similar idea would be to pick the population inside the perimeter instead of the area ; this would require some population data that could be challenging to get with accuracy.

Those techniques should prevent close stations from recording multiple times the same weather variation and should reward the important stations.

11.2.2 More features and data points

With enough computer power, it could be possible to directly take the station's data as a feature instead of aggregating it. The risk of overfitting would dramatically increase, as that represents 50 times the features for most regions. It would also imply properly filling the missing features.

We took all information available at the daily scale, but at the hourly scale we could still go back to 2013 ; that would also require more computer power to handle a dataset with over 100,000 rows.

Combining the two would create the most precise dataset possible and extensive computer power, but seeing the best result on such a dataset would be interesting.

12 Annexes

12.1 Feature names explained

12.1.1 Daily dataset

Here is a section of the description of the field provided by *Météo France* The original can be found here
NUM_POSTE : numéro Météo-France du poste sur 8 chiffres
NOM_USUEL : nom usuel du poste
LAT : latitude, négative au sud (en degrés et millionièmes de degré)
LON : longitude, négative à l'ouest de GREENWICH (en degrés et millionièmes de degré)
ALTI : altitude du pied de l'abri ou du pluviomà¨tre si pas d'abri (en m)
AAAAMMJJ : date de la mesure (année mois jour)
RR : quantité de précipitation tombée en 24 heures (de 06h FU le jour J à 06h FU le jour J+1). La valeur relevée à J+1 est affectée au jour J (en mm et 1/10)
TN : température minimale sous abri (en °C et 1/10)
HTN : heure de TN (hhmm)
TX : température maximale sous abri (en °C et 1/10)
HTX : heure de TX (hhmm)
TM : moyenne quotidienne des températures horaires sous abri (en °C et 1/10)
TNTXM : moyenne quotidienne (TN+TX)/2 (en °C et 1/10)
TAMPLI : amplitude thermique quotidienne : écart entre TX et TN quotidiens (TX-TN) (en °C et 1/10)

TNSOL : température quotidienne minimale à 10 cm au-dessus du sol (en °C et 1/10)
TN50 : température quotidienne minimale à 50 cm au-dessus du sol (en °C et 1/10)
DG : durée de gel sous abri (T $<$ 0°C) (en mn)
FFM : moyenne quotidienne de la force du vent moyenné sur 10 mn, à 10 m (en m/s et 1/10)
FF2M : moyenne quotidienne de la force du vent moyenné sur 10 mn, à 2 m (en m/s et 1/10)
FXY : maximum quotidien de la force maximale horaire du vent moyenné sur 10 mn, à 10 m (en m/s et 1/10)
DXY : direction de FXY (en rose de 360)
HXY : heure de FXY (hhmm)
FXI : maximum quotidien de la force maximale horaire du vent instantané, à 10 m (en m/s et 1/10)
DXI : direction de FXI (en rose de 360)
HXI : heure de FXI (hhmm)
FXI2 : maximum quotidien de la force maximale horaire du vent instantané, à 2 m (en m/s et 1/10)
DXI2 : direction de FXI2 (en rose de 360)
HXI2 : heure de FXI2 (hhmm)
FXI3S : maximum quotidien de la force maximale horaire du vent moyenné sur 3 s, à 10 m (en m/s et 1/10)
DXI3S : direction de FXI3S (en rose de 360)
HXI3S : heure de FXI3S (hhmm)
DRR : durée des précipitations (en mn)

12.1.2 Hourly dataset

The full document can be found here.
NUM_POSTE : numéro Météo-France du poste sur 8 chiffres
NOM_USUEL : nom usuel du poste
LAT : latitude, négative au sud (en degrés et millionièmes de degré)
LON : longitude, négative à l'ouest de GREENWICH (en degrés et millionièmes de degré)
ALTI : altitude du pied de l'abri ou du pluviomètre si pas d'abri (en m)
AAAAMMJJHH : date de la mesure (année mois jour heure)
RR1 : quantité de précipitation tombée en 1 heure (en mm et 1/10)
DRR1 : durée des précipitations (en mn)
FF : force du vent moyenné sur 10 mn, mesurée à 10 m (en m/s et 1/10)
DD : direction de FF (en rose de 360)
FXY : valeur maximale de FF dans l'heure (en m/s et 1/10)
DXY : direction de FXY (rose de 360)
HXY : heure de FXY (hhmm)
FXI : force maximale du vent instantané dans l'heure, mesurée à 10 m (en m/s et 1/10)

DXI : direction de FXI (en rose de 360)
HXI : heure de FXI (hhmm)
FF2 : force du vent moyenné sur 10 mn, mesurée à 2 m (en m/s et 1/10)
DD2 : direction de FF2 (en rose de 360)
FXI2 : force maximale du vent instantané dans l'heure, mesurée à 2 m (en m/s et 1/10)
DXI2 : direction de FXI2 (en rose de 360)
HXI2 : heure de FXI2 (hhmm)
FXI3S : force maximale du vent moyenné sur 3 secondes dans l'heure (en m/s et 1/10)
DXI3S : direction de FXI3S (en rose de 360)
HFXI3S : heure de FXI3S (hhmm)
T : température sous abri instantanée (en °C et 1/10)
TD : température du point de rosée (en °C et 1/10)
TN : température minimale sous abri dans l'heure (en °C et 1/10)
HTN : heure de TN (hhmm)
TX : température maximale sous abri dans l'heure (en °C et 1/10)
HTX : heure de TX (hhmm)
DG : durée de gel sous abri (T $<$ 0°C) (en mn)
T10 : température à 10 cm au-dessous du sol (en °C et 1/10), n'existe pas pour les sémaphores
T20 : température à 20 cm au-dessous du sol (en °C et 1/10), n'existe pas pour les sémaphores
T50 : température à 50 cm au-dessous du sol (en °C et 1/10), n'existe pas pour les sémaphores
T100 : température à 1 m au-dessous du sol (en °C et 1/10), n'existe pas pour les sémaphores
TNSOL : température minimale à 10 cm au-dessus du sol (en °C et 1/10)
TN50 : température minimale à 50 cm au-dessus du sol (en °C et 1/10)
TCHAUSSEE : température de surface mesurée sur herbe ou sur bitume (en °C et 1/10)
DHUMEC : durée d'humectation (en mn)
U : humidité relative (en UN : humidité relative minimale dans l'heure (en HUN : heure de UN (hhmm)
UX : humidité relative maximale dans l'heure (en HUX : heure de UX (hhmm)
DHUMI40 : durée avec humidité $<$ 40DHUMI80 : durée avec humidité $>$ 80TSV : tension de vapeur (en hPa et 1/10)
PMER : pression mer, seulement pour les postes dont l'altitude est inférieure ou égale à 750 m (en hPa et 1/10)
PSTAT : pression station (en hPa et 1/10)
PMERMIN : minimum horaire de la pression mer (en hPa et 1/10)
GEOP : géopotentiel, seulement pour les stations dont l'altitude est supérieure à 750 m (en mgp)
N : nébulosité totale (en octa), 9=ciel invisible par brouillard et/ou autre phénomène météorologique
NBAS : nébulosité de la couche nuageuse principale la plus basse (en octa)

CL : code SYNOP nuages bas (v. Atlas international des nuages (OMM-NÂ°407), Volume I, Classification des nuages, https ://cloudatlas.wmo.int/fr/code-specifications-and-coding-procedures.html)
CM : code SYNOP nuages moyens (v. Atlas international des nuages (OMM-NÂ°407), Volume I, Classification des nuages)
CH : code SYNOP nuages élevés (v. Atlas international des nuages (OMM-NÂ°407), Volume I, Classification des nuages)
N1 : nébulosité de la 1ère couche nuageuse (en octa)

Références

[1] Météo France. Données climatologiques de base - horaires. https://www.data.gouv.fr/datasets/donnees-climatologiques-de-base-horaires, 2026.

[2] Météo France. Données climatologiques de base - quotidiennes. https://www.data.gouv.fr/datasets/donnees-climatologiques-de-base-quotidiennes, 2026.

[3] RTE France. éco2mix - la consommation d'électricité en france. https://www.rte-france.com/donnees-publications/eco2mix-donnees-temps-reel/telecharger-indicateurs, 2026.

[4] RTE France. éco2mix - la consommation d'électricité en france. https://www.rte-france.com/donnees-publications/eco2mix-donnees-temps-reel/consommation-france, 2026.

[5] CAF OPEC Fund EIB The World Bank Group, ADB and EBRD. Statement on nuclear financing. https://www.elysee.fr/en/emmanuel-macron/2026/03/10/statement-on-nuclear-financing.